

Librerías JavaScript aplicadas al proyecto BackToMe

Kimberly Alexandra Rivera López (RL261989)

Andrea Michelle Jiménez Aguilar (JA262091)

Universidad Don Bosco

DAW901 G01T (Virtual): Desarrollo de Aplicaciones Web con Software Interpretados en el Cliente

Ciclo II – 2026

Alexander Alberto Siguenza Campos

21 de octubre de 2026

Tabla de contenido

Resumen 3

Introducción 4

Método de revisión 4

Dependencias identificadas 4

Express: rutas y middleware para la API 5

MySQL2: acceso asíncrono a MariaDB/MySQL 5

dotenv: configuración separada del código 6

cors: acceso del navegador a la API 6

Nodemailer: verificación del correo 7

Módulo nativo de Node.js usado junto a las librerías 7

Integración en BackToMe 7

Evaluación y alcance 8

Conclusiones 8

Referencias 10

Repositorio de GitHub: https://github.com/AguiJax2006-pixe/BackToMe-UDB.git

Link del video:

https://docs.google.com/videos/d/11rLTPCHv72OKDFwfiaWzqKsUZjWdigtUVZFvY5lw1tQ/play?usp=sharing

Resumen

Este documento analiza las dependencias JavaScript utilizadas en BackToMe, una aplicación de objetos perdidos y encontrados para la comunidad de la Universidad Don Bosco. A partir de la revisión del archivo *package.json* y del servidor *server.js*, se identifican Express, MySQL2, dotenv, cors y Nodemailer. Se explica cómo cada herramienta participa en la API, la conexión con MariaDB/MySQL, la configuración del entorno, el intercambio entre orígenes y la verificación de correo. También se describe el uso del módulo criptográfico nativo de Node.js, que no es una dependencia externa. La investigación relaciona la documentación oficial con decisiones observables del código y señala límites importantes: CORS no reemplaza la autorización, y las credenciales deben permanecer fuera del repositorio. El alcance se limita a las dependencias presentes en la versión revisada.

Palabras clave: JavaScript, Express, MySQL2, Nodemailer, BackToMe

Librerías JavaScript aplicadas al proyecto BackToMe

Introducción

BackToMe permite registrar usuarios, confirmar su correo, iniciar sesión y gestionar publicaciones, solicitudes y conversaciones relacionadas con objetos perdidos y encontrados. Para conectar la interfaz con la base de datos se incorporó un servidor Node.js. Este informe explica las librerías externas declaradas en el *package.json* del proyecto y las vincula con el uso observado en *server.js*. La interfaz se construye con HTML, CSS y JavaScript del navegador; este informe se enfoca en las dependencias del servidor, no en afirmar que el cliente usa una biblioteca de componentes.

Método de revisión

Se revisaron *package.json*, *server.js* y *.gitignore* del proyecto BackToMe, y se contrastó su uso con documentación oficial de los mantenedores de cada herramienta. Las versiones se presentan como rangos declarados por npm en el proyecto; no se infiere que el número indicado sea una versión fijada exactamente.

Dependencias identificadas

La Tabla 1 resume las dependencias externas declaradas en el *package.json* del proyecto, la versión indicada para cada una y el uso que se observó de ella en BackToMe.

Tabla 1

Dependencias externas declaradas en el proyecto BackToMe

Dependencia	Versión declarada	Uso observado en BackToMe
Express	^5.3.0	Rutas HTTP, middleware JSON y API
MySQL2	^3.24.6	Pool de conexiones y consultas asíncronas
dotenv	^18.0.8	Carga de configuración desde .env
cors	^2.8.6	Middleware de encabezados CORS
Nodemailer	^10.1.7	Envío SMTP del enlace de verificación

Nota. Los rangos de versión se transcribieron del *package.json* del proyecto consultado.

Express: rutas y middleware para la API

Express es el marco de aplicación del servidor. Su sistema de rutas permite asociar métodos HTTP, como GET, POST, PATCH y DELETE, con funciones que procesan solicitudes y generan respuestas (Express.js, s. f.-a). En BackToMe, *server.js* registra rutas para autenticación, perfil, objetos, solicitudes y mensajes. También usa *express.json()* para interpretar cuerpos JSON enviados por la interfaz (Express.js, s. f.-b). Esta estructura concentra la comunicación cliente-servidor y separa cada operación según su propósito.

La aplicación del marco es adecuada porque el proyecto necesita una API pequeña con rutas explícitas y middleware. Express no implementa por sí mismo la lógica de negocio ni la conexión SQL: esas tareas quedan en funciones del servidor y en MySQL2. Mantener esta distinción ayuda a revisar qué endpoint recibe cada acción y qué respuesta devuelve.

MySQL2: acceso asíncrono a MariaDB/MySQL

MySQL2 es un cliente para comunicarse con MySQL desde Node.js e incluye soporte para promesas, pools de conexiones y sentencias preparadas (Sidorares, s. f.-a). BackToMe importa la variante *mysql2/promise*, crea un pool y usa *execute()* con marcadores de posición para insertar y consultar datos. El pool limita conexiones simultáneas y permite reutilizarlas; el estilo con promesas facilita esperar resultados dentro de funciones asíncronas.

El uso de valores parametrizados separa los datos de la instrucción SQL; por ejemplo, la inserción de una publicación pasa los campos como arreglo de parámetros en vez de concatenarlos en la consulta. Esto reduce riesgos de inyección por esos valores y mejora la claridad. La documentación de MySQL2 describe *execute()* como una interfaz para ejecutar sentencias preparadas con parámetros (Sidorares, s. f.-b). La validación de entradas y la

autorización siguen siendo necesarias; una consulta parametrizada no comprueba por sí sola que quien realiza la acción sea el propietario del registro.

dotenv: configuración separada del código

dotenv carga las variables definidas en un archivo .env y las agrega a *process.env* para que Node.js las use durante la ejecución (Motdotla, s. f.). En BackToMe, esto permite configurar host, puerto, usuario y contraseña de la base, además de la cuenta SMTP y la URL base. La aplicación evita escribir directamente esos valores en las consultas o en el transporte de correo.

Separar configuración del código facilita ejecutar el proyecto en distintos equipos, pero .env no cifra los valores. Debe mantenerse fuera de Git; el .gitignore del proyecto excluye ese archivo y el README indica que no se debe subir. Para compartir el proyecto, se usa .env.example sin credenciales reales y cada integrante configura su propia copia local.

cors: acceso del navegador a la API

El paquete cors agrega encabezados de intercambio de recursos entre orígenes (CORS) a las respuestas del servidor. El navegador usa esos encabezados para decidir si el código JavaScript de otro origen puede leer la respuesta (Goode, s. f.). En *server.js* se registra *app.use(cors())*, habilitando la política predeterminada del paquete.

La configuración actual es práctica para desarrollo local, pero la documentación advierte que CORS no autentica usuarios ni bloquea clientes que no sean navegadores. Por tanto, BackToMe debe seguir protegiendo sus rutas con sesiones y verificaciones de propiedad; al publicar la API se recomienda reemplazar el origen abierto por la lista de orígenes necesarios (Goode, s. f.).

Nodemailer: verificación del correo

Nodemailer ofrece una interfaz de Node.js para crear un transporte y enviar mensajes por SMTP u otros servicios (Nodemailer, s. f.). BackToMe configura un transporte Gmail con credenciales de aplicación provenientes de variables de entorno. Al registrarse, crea un token aleatorio, almacena un hash del token con vencimiento y envía al correo indicado un enlace de confirmación. La cuenta queda pendiente hasta que el enlace sea validado. El mensaje se genera en formato de texto y HTML.

Esta integración aporta una comprobación de posesión de la dirección de correo y permite activar el flujo de inicio de sesión después de confirmar la cuenta. El envío requiere conexión, configuración SMTP y manejo de fallos; no sustituye controles contra abuso ni la protección de las credenciales del remitente.

Módulo nativo de Node.js usado junto a las librerías

El servidor también utiliza el módulo nativo *crypto* de Node.js, que no es una dependencia npm adicional. El código genera tokens aleatorios, guarda un hash SHA-256 del token de verificación y deriva hashes de contraseña mediante scrypt, comparándolos con una función de tiempo constante. Esta decisión aprovecha APIs incluidas en el entorno de ejecución, por lo que conviene distinguirlas de Express, MySQL2, dotenv, cors y Nodemailer en el inventario del proyecto (OpenJS Foundation, s. f.).

Integración en BackToMe

El recorrido principal conecta las dependencias de esta manera: el navegador envía datos JSON; Express recibe la petición y elige una ruta; la ruta valida los datos y usa MySQL2 para leer o actualizar MariaDB. dotenv entrega la configuración necesaria al proceso Node.js.

Cuando el registro requiere confirmar el correo, Nodemailer entrega el enlace por Gmail. CORS participa en las respuestas que debe leer el navegador si la interfaz y la API se sirven desde orígenes distintos. La protección efectiva depende también de la lógica de sesiones, permisos, validación y consultas del servidor, no de una librería aislada.

Evaluación y alcance

La combinación responde a necesidades concretas del proyecto: API HTTP, persistencia compartida, configuración por equipo y verificación de correo. El código observado usa parámetros SQL, hash de contraseñas, tokens de un solo uso y comprobaciones de sesión y pertenencia en rutas privadas. Son decisiones importantes para un prototipo, aunque antes de un despliegue real se debería restringir CORS, forzar HTTPS, definir rotación y almacenamiento seguro de secretos, revisar límites de solicitudes y probar los flujos de error.

El *package.json* revisado no declara Chart.js ni SweetAlert, y los archivos inspeccionados tampoco muestran referencias a esas bibliotecas. Por eso este informe no las describe como implementadas. Si la gráfica exigida por la rúbrica aún no está incorporada, su implementación y validación con datos del proyecto siguen siendo una tarea independiente; este documento no sustituye ese requisito.

Conclusiones

BackToMe combina bibliotecas con responsabilidades delimitadas. Express organiza la API, MySQL2 conecta las rutas con MariaDB, dotenv mantiene la configuración fuera del código, cors regula los encabezados que el navegador necesita y Nodemailer implementa el correo de verificación. El módulo nativo crypto apoya la gestión de tokens y contraseñas sin añadir una dependencia externa. La revisión del código y su comparación con documentación oficial muestran una relación directa entre las herramientas elegidas y los flujos del proyecto.

Para la siguiente etapa, la seguridad debe reforzarse en la configuración de producción y la entrega debe demostrar los módulos mediante datos y escenarios reales.

Referencias

Express.js. (s. f.-a). *Routing*. https://expressjs.com/en/5x/guide/routing/

Express.js. (s. f.-b). *Using middleware*. https://expressjs.com/guide/using-middleware/

Goode, T. (s. f.). *cors* [Software]. GitHub. https://github.com/expressjs/cors

Motdotla. (s. f.). *dotenv* [Software]. GitHub. https://github.com/motdotla/dotenv

Nodemailer. (s. f.). *Nodemailer documentation*. https://nodemailer.com/

OpenJS Foundation. (s. f.). *Crypto*. Node.js. https://nodejs.org/api/crypto.html

Sidorares. (s. f.-a). *Documentation*. MySQL2. https://sidorares.github.io/node-mysql2/docs/documentation

Sidorares. (s. f.-b). *Prepared statements*. MySQL2. https://sidorares.github.io/node-mysql2/docs/documentation/prepared-statements